

Rapport du mini-projet Candy Crush

I. Niveau de difficulté implémenté

Les deux niveaux de difficultés ont été implémentés, le choix est possible lors du lancement du jeu.

Le jeu lance alors un menu sur lequel vous pouvez choisir le niveau (voir Moodle), le nombre de bonbons entre 3 et 7 ainsi que le choix d'import d'un fichier ou non.

NB : Le fichier csv choisi devra être nommé "exemple_grille.csv" pour être reconnu par le programme et être correctement lancé.

II. Règles de jeu et choix effectués (en quelques lignes)

Les règles du jeu sont celles du CandyCrush basique, pour lancer le programme principal, il faut bien **télécharger l'ensemble du dossier** sur son ordinateur, et exécuter le fichier "menu_mat.py".

III. Algorithme principal en pseudo-code

L'algorithme présenté ci-dessous décrit le fonctionnement global de notre code mais ne correspond pas à une partie unique du code, car celui-ci est composé d'une multitude de fonctions non triviales servant notamment à afficher le jeu, et qui ne sont donc pas intéressantes dans le cas présent :

sélection du mode de jeu

création ou récupération de la grille

TÂCHE 1

tant qu'on peut jouer (il y a des coups possibles):

TÂCHE 5

afficher le jeu

TÂCHE 2

si la sélection du joueur est valide :

TÂCHE 6

échanger les pions concernés

trouver tous les pions à supprimer

TÂCHE 3

supprimer ces pions

TÂCHE 4

faire tomber les bonbons du dessus

tant qu'il reste des pions à supprimer "en cascade" :

afficher le jeu

TÂCHE 2

trouver les pions à supprimer

TÂCHE 3

les supprimer

TÂCHE 4

faire tomber les bonbons du dessus

mettre à jour le score

IV. Description de l'archive de tests

Pour la détection des alignements, décrivez le contenu de l'archive et les tests effectués.

```
# Jeu de test pour test alignement :  
  
# #1. Rien à supprimer :  
  
# Entrée : [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]  
  
# Sortie : []  
  
#  
  
# #2. Tout à supprimer :  
  
# Entrée : [[0, 0, 0, 0], [0, 0, 1, 1], [0, 0, 0, 1], [0, 1, 1, 1]]  
  
# Sortie : [(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 1), (1, 2), (1, 3), (2, 0), (2, 1),  
(2, 2), (2, 3), (3, 0), (3, 1), (3, 2), (3, 3)]  
  
#  
  
# #3. Une partie à supprimer :  
  
# Entrée : [[0, 1, 2, 3], [0, 0, 6, 7], [0, 0, 10, 11], [12, 0, 0, 0]]  
  
# Sortie : [(0, 0), (1, 0), (1, 1), (2, 0), (2, 1), (3, 1), (3, 2), (3, 3)]
```

V. Complexité d'une tâche

Complexité de "test_alignement" :

- 2 boucles for : $O(n*m)$ (ou $O(n^2)$)
- while : complexité indéfinie (dépend fortement de la grille) mais systématiquement inférieure à n^2
- Somme des 2 : $O(n^2)$ (ou $O(n*m)$)